



Securing Apache

In the previous article in this series, we started our journey to a secured Apache by dissecting its internals. We then looked at various attacks against Web applications via injection flaws, beginning with SQL injection. In this article, we will deal with another category of injection flaws: Cross-Site Scripting, a.k.a. 'XSS'. I would like to reiterate here that neither I nor LFY aim to teach readers to attack servers; this is meant to give you the knowledge that you need to protect your own infrastructure.

Grabbing second position in OWASP's latest Top Ten critical Web application security risks—after SQL injection flaws—is XSS. (By the way, the different first letter is used to avoid confusion with CSS—Cascading Style Sheets.) The security consortium says that XSS accounts for about 39 per cent of vulnerabilities in Web applications.

So what is XSS?

OWASP defines XSS as a flaw that occurs when an application includes user-supplied data in a page sent back to the browser, without properly validating or escaping that data. XSS attacks are essentially code-injection attacks, which exploit the interpretation process of the Web application in the browser. These attacks are carried out mainly on online message boards, Web logs, guest books, and user forums (collectively called 'boards', in the rest of the article), where messages

are permanently stored. They are created using HTML, JavaScript, VBScript, ActiveX, Flash, and other client-side scripting technologies.

The goal of an XSS attack is to steal client authentication cookies, and any other sensitive information that can authenticate the client to the website. With a captured (legitimate) user token, an attacker can impersonate the user, leading to identity theft.

Unlike most attacks, which involve two parties (the attacker and the website, or the attacker and the victim/client), the XSS attack involves three parties: the attacker, the victim/client, and the website. An XSS attack tricks a legitimate user by posting a message to the board with a link to a seemingly harmless site, which subtly encodes a script that attacks the users once they click the link. This seemingly harmless website can be (and is, in many cases) a phishing clone of a page in the original website the user is browsing; it may